ClockBot 浏览器自动化与人工操作的技术审计

1. 文档目的

这份文档说明两件事：

1. `ClockBot` 当前是如何打开浏览器并完成 IEYASU 登录、出勤、退勤操作的。
2. 从网站角度观察，程序自动打开浏览器并完成点击操作，与人工手动打开浏览器并完成相同操作，在技术上可以如何区分。

本文只讨论判别信号、证据强弱、当前实现可能暴露的观察面，不讨论绕过检测、隐藏自动化或伪装人工行为的方法。

2. 当前实现摘要

根据当前源码，`ClockBot` 的自动化流程不是“调用系统默认浏览器，让用户当前正在使用的浏览器自己工作”，而是：

1. 使用 `playwright-core` 启动本机安装的 `Chrome / Edge / Chromium`。
2. 使用 `ClockBot` 自己的持久化 profile 目录，而不是用户日常浏览器的默认 profile。
3. 默认以无头模式运行，只有 `showBrowser` 为 `true` 时才显示窗口。
4. 直接跳转到登录地址。
5. 使用 `fill()` 写入用户名和密码。
6. 在页面上下文中执行 DOM `element.click()` 点击登录按钮和出勤/退勤按钮。
7. 在 Windows 上先尝试读取系统定位，再向浏览器上下文授予地理位置权限并注入坐标。
8. 通过页面文本、控件文字、按钮面积与颜色等启发式规则判断按钮是否可操作。
9. 在操作完成或失败后关闭这个自动化浏览器上下文。

代码可确认

- `playwright-core` 的 `chromium.launchPersistentContext(...)` 用于启动浏览器上下文，且当前只显式传入了 `executablePath` 与 `headless` 选项，见 `src/services/ieyasu-automation.js:675-678`。
- 浏览器可执行文件通过 `CLOCKBOT_BROWSER_PATH` 或预设 Chrome / Edge / Chromium 路径查找，见 `src/services/ieyasu-automation.js:46-85`。
- 持久化 profile 固定为 `ClockBot/automation-profile`，见 `src/services/ieyasu-automation.js:88-101`。
- 默认 `showBrowser` 为 `false`，见 `src/services/settings-service.js:6-12`。
- 登录字段通过 `fill()` 填充，见 `src/services/ieyasu-automation.js:651-652`。
- 登录与出勤/退勤按钮通过页面内 `target.click()` 触发，见 `src/services/ieyasu-automation.js:147-179` 与 `src/services/ieyasu-automation.js:447-483`。
- Windows 定位通过 `GeoCoordinateWatcher` 获取，再传给自动化上下文，见 `src/services/system-location-service.js:42-79` 与 `src/services/ieyasu-automation.js:681-684`。

3. 总览表

维度	程序当前行为	人工通常行为	主要影响	代码依据
浏览器启动来源	Playwright 启动本机 Chrome / Edge / Chromium	用户自己打开某个浏览器或某个现有窗口	启动上下文与控制通道不同	`src/services/ieyasu-automation.js:46-85`, `:675-678`
浏览器 profile	使用 `ClockBot/automation-profile`	使用用户日常 profile	Cookie、Local Storage、权限、扩展、历史状态隔离	`src/services/ieyasu-automation.js:88-101`
是否显示窗口	默认无头，`showBrowser=true` 才显示	基本总是可见窗口	事件轨迹、权限流程、用户介入路径不同	`src/services/settings-service.js:6-12`, `:91-93`; `src/services/ieyasu-automation.js:675-678`
登录输入方式	`fill()` 直接写入输入框	键盘逐步输入	输入事件链与节奏不同	`src/services/ieyasu-automation.js:651-652`
点击方式	页面脚本内 `target.click()`	鼠标移动、悬停、按下、抬起	点击前后的前端事件序列不同	`src/services/ieyasu-automation.js:147-179`, `:447-483`
定位处理	Windows 下授予地理位置权限并注入系统坐标	常见情况是浏览器自己弹权限并按用户路径返回定位	权限状态与定位返回模式不同	`src/services/system-location-service.js:42-79`; `src/services/ieyasu-automation.js:681-684`
会话判定	登录表单不存在时视为 session 已存在	人工依据当前浏览器真实登录状态理解页面	自动化 profile 的登录态与人工 profile 可能不同步	`src/services/ieyasu-automation.js:643-649`
出勤/退勤判定	依据按钮文字、面积、颜色亮度推断 active/inactive	人工按语义理解页面	页面微调后程序与人工判断可能不同	`src/services/ieyasu-automation.js:234-445`
等待节奏	固定超时、固定轮询、固定状态转换确认	人工节奏更不稳定且会临场调整	时间分布更像脚本	`src/services/ieyasu-automation.js:489-595`

维度	程序当前行为	人工通常行为	主要影响	代码依据
生命周期	完成后关闭自动化 context	人工常继续停留或切换标签页	会话持续时间与后续行为差异明显	`src/services/ieyasu-automation.js:767-768`

4. 程序视角 vs 人工视角

本节按固定结构对比当前实现与人工操作。

4.1 浏览器实例来源

程序当前怎么做

程序先在本机查找 Chrome、Edge、Chromium 可执行文件，再通过 `chromium.launchPersistentContext(...)` 拉起受 Playwright 控制的浏览器上下文。

人工通常怎么做

人工一般通过桌面图标、任务栏、开始菜单、浏览器已打开窗口、书签或历史记录进入目标网站。

差异产生原因

程序依赖自动化框架控制浏览器；人工依赖操作系统与浏览器原生交互链路。

实际影响

即使外观上都可能是 Chrome 或 Edge，网站前端与风控系统看到的运行环境、会话连续性、自动化可探测性和用户交互上下文都可能不同。

代码依据

`src/services/ieyasu-automation.js:46-85`, `src/services/ieyasu-automation.js:675-678`

4.2 Profile、Cookie 与站点状态

程序当前怎么做

程序固定使用 `ClockBot/automation-profile` 作为持久化 profile 目录。

人工通常怎么做

人工操作时通常使用自己日常浏览器的默认 profile 或某个长期使用的个人 profile。

差异产生原因

自动化为了稳定复用自己的站点状态，需要单独的持久化目录；人工使用的是个人长期环境。

实际影响

即使是同一台机器、同一个浏览器品牌，网站观察到的 cookie、localStorage、sessionStorage、权限记忆、扩展存在与否、登录态连续性，也可能与人工浏览器完全不同。

代码依据

`src/services/ieyasu-automation.js:88-101`

4.3 可见窗口与无头模式

程序当前怎么做

默认 `showBrowser=false`，因此自动化默认以无头方式运行；只有显式打开 `showBrowser` 才显示窗口。

人工通常怎么做

人工基本总是在有可见窗口的前提下操作。

差异产生原因

自动化更关注计划任务稳定性和后台运行；人工则依赖可见界面反馈。

实际影响

权限提示、视觉等待、鼠标轨迹、焦点切换与用户中断路径都可能不同。网站如果有前端埋点，也更容易观察到“无明显人工界面交互痕迹”的会话。

代码依据

`src/services/settings-service.js:6-12`, `src/services/settings-service.js:91-93`, `src/services/ieyasu-automation.js:675-678`

4.4 进入登录页的方式

程序当前怎么做

程序直接 `goto(loginUrl)` 导航到登录页。

人工通常怎么做

人工可能从书签、历史记录、浏览器自动补全、收藏夹、公司门户、上一次会话恢复页等路径进入。

差异产生原因

自动化会追求最短路径和最稳定入口；人工来源更分散。

实际影响

页面来源、停留节奏、跳转链路和前序页面上下文更像脚本直接落点，而不像真实用户在浏览器中自然导航。

代码依据

`src/services/ieyasu-automation.js:624-628`

4.5 账号密码输入方式

程序当前怎么做

程序调用 `fill()` 直接写入用户名和密码。

人工通常怎么做

人工通过键盘逐步输入，可能伴随停顿、纠错、焦点切换、剪贴板粘贴、输入法状态变化等过程。

差异产生原因

`fill()` 更像直接修改输入框值并触发必要事件，而不是完整复刻人类逐键输入过程。

实际影响

如果网站在前端采集输入时序，它可能观察到：

- 两个字段值出现得很快。
- 按键级间隔不符合人类打字节奏。
- 缺少自然的按键分布、删除、回退与重新聚焦行为。

代码依据

`src/services/ieyasu-automation.js:651-652`

4.6 点击方式

程序当前怎么做

程序在页面上下文里找到目标元素，然后执行 `target.click()`。

人工通常怎么做

人工通常会经历鼠标移动、悬停、命中目标、`mousedown`、`mouseup`、`click`，或者以键盘导航触发控件。

差异产生原因

程序当前并没有通过 Playwright 的真实指针操作接口去移动鼠标，而是直接触发 DOM 层点击。

实际影响

网站前端如果监听较细的交互事件链，就可能观察到：

- 点击前缺少真实指针移动轨迹。
- 缺少 hover 或 hover 链路过短。
- 事件来源更像程序触发而不是设备输入触发。

代码依据

`src/services/ieyasu-automation.js:147-179`, `src/services/ieyasu-automation.js:447-483`

4.7 登录态判定

程序当前怎么做

如果页面上没有找到登录表单，程序直接认为“已有有效 session”，然后继续往后执行。

人工通常怎么做

人工会根据自己常用浏览器里真实展示的页面语义判断自己是否已登录。

差异产生原因

自动化依赖 DOM 结构和 profile 内已有状态；人工依赖视觉理解和长期使用环境。

实际影响

人工浏览器已登录，并不等于自动化 profile 也已登录；反过来也一样。网站看到的是 `ClockBot` 这个独立 profile 的真实会话状态，而不是用户主浏览器的状态。

代码依据

`src/services/ieyasu-automation.js:643-649`

4.8 出勤/退勤按钮可操作性判断

程序当前怎么做

程序先提取可见交互控件，再根据文字、面积、背景色、前景色、亮度与填充得分推断 `clockIn` 与 `clockOut` 哪个按钮处于 `active` 或 `inactive` 状态。

人工通常怎么做

人工主要依赖页面语义、标签文本、视觉布局与上下文理解来判断当前应该点击哪个按钮。

差异产生原因

自动化需要把“人眼理解”简化为规则；当前实现选择了颜色与文本启发式，而不是业务语义 API。

实际影响

网站稍微改版、改色、换样式、加入更多按钮或调整文案后，程序与人工可能对“当前是否可点”得出不同结论。

代码依据

`src/services/ieyasu-automation.js:234-445`

4.9 等待、轮询与节奏

程序当前怎么做

程序会：

- 登录后等待 `networkidle` 或导航完成。
- 每 1 秒轮询页面状态。
- 最长等待 90 秒让出勤按钮变成可操作状态。
- 点击后最多等待 15 秒确认状态切换。

人工通常怎么做

人工的等待时间、停顿长度、是否刷新、是否回退、是否多看几眼页面，都很不稳定。

差异产生原因

程序使用固定超时和轮询逻辑；人工行为天然更噪声化。

实际影响

如果网站记录详细行为时间线，自动化的时间分布会更规律、更窄，且在多次运行之间更容易重复。

代码依据

`src/services/ieyasu-automation.js:489-595`

4.10 定位权限与地理位置来源

程序当前怎么做

在 Windows 上，程序先从系统定位服务读取坐标，再对目标站点 origin 授予 `geolocation` 权限，并把坐标设置到浏览器上下文里。非 Windows 平台则依赖浏览器自身的定位权限流程。

人工通常怎么做

人工常见路径是由浏览器向用户展示定位权限提示，再由用户同意或拒绝，然后浏览器自行向网站返回定位结果。

差异产生原因

当前实现希望尽量减少 Windows 上第一次可见运行时的人工介入。

实际影响

网站虽然看不到“程序调用了哪个 API 授权”，但它可以观察权限状态、定位请求返回速度、坐标稳定性、是否经历常见授权路径，以及是否与账户历史模式一致。

代码依据

`src/services/system-location-service.js:42-79`, `src/services/ieyasu-automation.js:681-684`

4.11 浏览器生命周期

程序当前怎么做

无论成功还是失败，程序都会在最后关闭浏览器上下文。

人工通常怎么做

人工常常会继续停留页面、查看结果、切换标签页、打开其他网站或保持浏览器持续运行。

差异产生原因

自动化把浏览器当作单次任务执行容器；人工把浏览器当作长期工作环境。

实际影响

单次访问时长、后续导航链路、同一会话的附加行为，往往与人工使用模式不同。

代码依据

`src/services/ieyasu-automation.js:767-768`

5. 从网站角度如何技术判别

这一节讨论的是“网站理论上能看到什么”和“当前实现最可能暴露什么”。为了避免把判别能力写得过满，本文使用以下四个标记：

- **代码可确认**：当前仓库源码直接能证明的事实。
- **网站前端通常可以观测**：只要页面中有相应前端脚本或埋点，浏览器端通常能观察到的信号。
- **网站服务端可以结合埋点与历史关联推断**：需要服务器接收埋点、日志或行为时间线后进行关联分析。
- **只能作为推断，不能从当前源码直接断言**：从通用浏览器/自动化知识可做出的合理推断，但当前仓库没有显式实现或显式声明。

5.1 浏览器与会话层

网站可观察到什么

- 当前 cookie、session、localStorage、sessionStorage 的内容。
- 当前站点权限状态，例如该 origin 是否已被授予某些权限。
- 浏览器实例中的会话连续性，例如是不是一个刚建立的站点环境。
- 前端脚本可读取到的基础环境属性，例如 `navigator`、窗口尺寸、语言环境、时区表现、权限状态、插件/扩展侧特征等。

当前实现会暴露什么

代码可确认

- 当前实现使用独立 profile，而不是用户平时的主浏览器 profile，见 `src/services/ieyasu-automation.js:88-101`。
- 当前实现启动浏览器上下文时，源码中只显式设置了 `executablePath` 与 `headless`，见 `src/services/ieyasu-automation.js:675-678`。

只能作为推断，不能从当前源码直接断言

- 由于没有看到对 `userAgent`、`viewport`、`locale`、`timezone`、启动参数的显式覆盖，网站前端若探测这些字段，看到的将是 Playwright 拉起浏览器后的默认暴露面，而不是用户长期手工维护的个人环境。
- 独立 profile 会让站点更容易观察到与用户常用浏览器环境不一致的 session 连续性、权限记忆、扩展生态和历史状态。

网站如何技术判别

- 如果同一账号在“人工主浏览器”里长期有稳定 cookie 与站点状态，而某些打卡请求却总来自一个更“新”、更干净、历史足迹更薄的浏览器环境，这本身就是明显差异。
- 如果网站在前端采集浏览器环境指纹，再与账号历史、最近活跃设备或同日其他访问会话比对，也能发现“像是另一个浏览器环境”。

5.2 前端事件层

网站可观察到什么

前端脚本若监听输入框、按钮和页面交互事件，通常可以看到：

- `focus` / `blur`
- `keydown` / `keypress` / `keyup`
- `beforeinput` / `input` / `change`
- `mousemove` / `mouseover` / `mouseenter`
- `mousedown` / `mouseup` / `click`
- 焦点切换顺序
- 事件间隔与时间戳分布

当前实现会暴露什么

代码可确认

- 用户名和密码通过 `fill()` 写入，见 `src/services/ieyasu-automation.js:651-652`。
- 登录按钮通过页面内 `target.click()` 触发，见 `src/services/ieyasu-automation.js:147-179`。
- 出勤/退勤按钮同样通过页面内 `target.click()` 触发，见 `src/services/ieyasu-automation.js:447-483`。

网站前端通常可以观测

- 输入框值变化出现得很快，且更像“字段被一次性填入”而不是人逐键打字。
- 点击前缺少自然鼠标移动和悬停链路。
- 点击事件的上下文更接近脚本触发的 DOM 点击，而不是完整的人机设备输入路径。

网站如何技术判别

- 如果登录字段在极短时间内同时完成填充，而没有自然的键盘事件分布，这更像自动化写值。
- 如果按钮点击之前没有充足的指针移动轨迹、hover 停留或对齐误差，点击路径更像脚本直接命中元素。
- 如果同一类动作多次重复时，事件顺序和时间间隔高度稳定，也更像程序而不是人。

5.3 页面行为与时序层

网站可观察到什么

只要网站有前端埋点或服务端日志，它就能对以下信息建立时间线：

- 进入页面的时间
- 首次可交互时间
- 输入开始与结束时间
- 登录提交时间
- 到达打卡页面的时间
- 出勤/退勤按钮点击时间
- 请求发出与成功返回时间

当前实现会暴露什么

代码可确认

- 登录后会等待 `networkidle` 或导航完成，见 `src/services/ieyasu-automation.js:584-595`。
- 程序每 1 秒轮询按钮状态，最多等待 90 秒，见 `src/services/ieyasu-automation.js:519-582`。
- 点击后最多等待 15 秒确认状态切换，见 `src/services/ieyasu-automation.js:489-517`。

网站服务端可以结合埋点与历史关联推断

- 多次成功打卡之间可能出现相似的访问节奏和时间分布。
- 从打开页面到填充凭据、从到达页面到点击目标按钮，这些阶段性延迟可能比人工更稳定。

网站如何技术判别

- 单次会话里，时间线过短、过稳、缺少犹豫和回退，不一定能直接定性为自动化，但会提高怀疑度。
- 多次会话叠加后，如果登录到点击的延迟分布高度集中、行为路径几乎一致，就更适合作为自动化风险信号。

5.4 权限与定位层

网站可观察到什么

- 当前 origin 的 `geolocation` 权限状态。
- 页面请求定位时的返回速度、是否超时、坐标是否可用。
- 坐标是否与该账号历史、IP 地理位置或设备历史一致。

当前实现会暴露什么

代码可确认

- Windows 上会读取系统定位，见 `src/services/system-location-service.js:42-69`。
- 自动化上下文会对目标 origin 授予 `geolocation` 权限，并在有坐标时设置地理位置，见 `src/services/ieyasu-automation.js:681-684`。
- 非 Windows 平台依赖浏览器自身权限流程，见 `src/services/system-location-service.js:72-79`。

网站前端通常可以观测

- 首次调用定位 API 时，权限状态和返回模式可能与人工授权路径不同。

网站服务端可以结合埋点与历史关联推断

- 坐标返回极快且模式稳定时，更像预先已具备定位能力的自动化环境。
- 同账号若长期呈现高度一致的坐标、授权结果和打卡节奏，也会提高可疑度。

网站如何技术判别

- 网站看不到 Playwright 的 `grantPermissions()` 调用本身，但能看到权限结果与定位结果。
- 结合前端埋点，网站可以区分“用户刚刚手动同意权限后返回定位”与“页面请求一到就已有可用权限/可用坐标”的差异。

5.5 服务端关联与风控层

服务端直接能看到什么

- 登录请求和打卡请求本身。
- IP 地址、TLS 会话、请求头、时间戳、账号、服务端 session。
- 同一账号的历史成功与失败记录。

需要前端埋点辅助才能看到什么

- 输入与点击的细粒度时序。
- 鼠标路径、hover 行为、焦点切换。
- 页面级权限状态和前端环境细节。

当前实现会暴露什么

代码可确认

- 自动化流程会按照较固定的控制流执行：进入登录页、判断是否已登录、等待按钮可操作、点击、确认状态切换，见 `src/services/ieyasu-automation.js:624-768`。

网站服务端可以结合埋点与历史关联推断

- 相同账号多次出现近似相同的登录到点击延迟。
- 会话环境与人工常用设备不连续。
- 位置返回、点击成功路径与时间分布高度一致。

网站如何技术判别

- 单个信号通常不够下结论，服务端更常用的是“弱信号叠加打分”。
- 当账号历史、浏览器环境、前端事件链、定位模式与请求时序同时偏向脚本特征时，判别就会更可靠。

6. 证据强度分层

下表按“从网站视角的证据力度”整理信号。这里的“强弱”不是法律意义上的证据强度，而是用于工程判断的风控价值。

信号	证据类型	网站是否能直接看到	是否适用于当前实现	单独使用是否足够	说明
独立 profile 导致的站点状态与用户常用环境不一致	强启发式证据	不能直接看到 profile 路径，但能看到其后果	是	否	网站看不到 `ClockBot/automation-profile` 路径本身，但能看到 cookie、权限、存储状态与账号历史不连续
输入框值被快速填充，缺少自然键盘节奏	强启发式证据	需要前端埋点	是	否	`fill()` 很容易在时序上与人工打字分离

信号	证据类型	网站是否能直接看到	是否适用于当前实现	单独使用是否足够	说明
点击前缺少鼠标移动、hover、按下/抬起链路	强启发式证据	需要前端埋点	是	否	当前点击来自页面脚本的 `element.click()`
浏览器环境呈现自动化可探测特征	直接证据或强启发式证据	前端通常可见	可能	视站点探测能力而定	当前仓库未做环境伪装；站点若做自动化探测，可能拿到高价值信号
Windows 下定位返回模式更像预授权/预注入环境	强启发式证据	前端与服务端都可部分感知	是	否	网站能看到权限结果与坐标返回，但看不到 Playwright API 调用本身
打开页面到登录、登录到点击的延迟分布高度稳定	强启发式证据	需日志或埋点	是	否	单次不稳定，多次叠加后价值上升
每秒轮询带来的规则性页面访问节奏	弱启发式证据	部分可见	是	否	更适合和其他信号组合使用
出勤/退勤操作总沿着同一成功路径完成	弱启发式证据	服务端可见	是	否	人工也可能熟练，但长时间完全一致更像程序
页面未出现人工常见纠错、回退、犹豫动作	弱启发式证据	需前端埋点	是	否	单独不足以定性

7. 本项目当前最可能被网站感知的 5 个信号

以下按当前实现的相关性排序。

1. **独立 profile 带来的会话环境差异** 网站无法直接看到本地目录名，但能看到与用户常用浏览器不一致的 cookie、localStorage、权限状态、历史连续性。
2. `fill()` **带来的非人工输入节奏** 只要网站有输入事件埋点，就很容易看出“值是很快被填进去的”，而不是自然打字。
3. `element.click()` **带来的非真实指针事件链** 当前实现没有真实鼠标移动路径，点击链路更像脚本直接命中元素。
4. **自动化控制上下文本身的可探测性** 当前仓库没有看到对常见环境字段的主动覆盖；如果站点前端有自动化探测，它有机会直接拿到高价值信号。

5. **Windows 下定位权限与定位返回模式差异** 网站看不到底层授权 API，但能看到定位权限结果、坐标返回速度与历史稳定性。

8. 服务器直接可见 vs 必须依赖前端埋点

为了避免混淆，下面把网站视角再拆一次。

8.1 服务器直接可见

- 登录请求与打卡请求本身
- 时间戳
- IP 与网络层上下文
- 账号与 session
- 请求成功/失败路径
- 同账号的历史模式

8.2 前端脚本通常可见

- 输入事件顺序
- 鼠标移动与 hover
- 焦点切换
- 点击事件上下文
- 页面权限状态
- 浏览器环境属性
- 页面停留和交互时间线

8.3 必须前后端联合才能更可靠

- 将“环境指纹 + 输入节奏 + 点击事件链 + 定位模式 + 账号历史”叠加成风险评分
- 将单次访问的弱信号提升为长期稳定模式
- 区分“偶然一次动作快”与“长期自动化重复路径”

9. 需要特别注意的边界

1. 网站通常不会仅凭一个信号就断言“这是脚本”。
2. 单次会话中很多现象只能算启发式怀疑，真正可靠的是多信号叠加。
3. 网站未必具备足够强的前端埋点或风控能力；“理论上可观测”不等于“该站点一定已实现”。
4. 当前仓库可以直接证明的是浏览器启动方式、profile 分离、`fill()`、`element.click()`、定位注入、轮询等待和关闭上下文；其他关于站点判别能力的描述都应理解为通用技术路径，而不是对目标网站现状的断言。

10. 结论

从技术上说，`ClockBot` 当前实现与人工手动打开浏览器并进行相同点击操作，最本质的差异不在“页面最后收到的那个业务请求是不是一样”，而在以下几层：

1. 浏览器与会话环境不是同一个长期用户环境。
2. 输入与点击事件链不是同一种人机交互来源。
3. 定位权限与定位结果的产生路径不同。

4. 整体时间线更规律、更接近脚本控制流。
5. 这些差异一旦被前端埋点和服务端历史关联起来，就足以让网站以较高概率把自动化操作与人工操作区分开来。

如果只看单个 HTTP 请求，自动化与人工未必容易完全区分；如果看前端事件、权限状态、定位模式、会话环境与长期行为轨迹，区分难度会明显下降。